

Financial Vault

Tools Used & Development Phases

Project 11 · Information Security · Faculty of Engineering

Part 1 — Tools & Technologies

The following tools and libraries were selected based on their maturity, security track record, and suitability for a self-contained academic project.

Tool / Library	Version	Role & Justification
Python	3.12	Primary language. Chosen for its rich ecosystem, readable syntax, and availability of security-focused libraries.
Flask	3.x	Lightweight WSGI micro-framework. Handles HTTP routing, Jinja2 templating, and form validation.
PyCryptodome	3.x	Provides AES, 3DES, ChaCha20, PBKDF2, and random byte generation. C-backed for performance.
Jinja2	(bundled with Flask)	Server-side template engine. Used for template inheritance (base.html) and dynamic content rendering.
SQLite / sqlite3	(stdlib)	Zero-configuration embedded database. Stores the audit_log table. No separate database server required.
hashlib	(stdlib)	SHA-256 computation for integrity fingerprinting. Part of Python's standard library.
struct	(stdlib)	Binary packing and unpacking of the .enc bundle header (algorithm length, IV length, etc.).
HTML5 / CSS3 / JS	—	Frontend. Custom CSS palette (Berry & Cream theme). Vanilla JS for toast notifications and form handling.
Visual Studio Code	—	Development IDE. Python and Jinja2 extensions used for syntax highlighting and linting.
Git	—	Version control. Used to track changes across development phases.

Part 2 — Development Phases

The project was developed iteratively across five phases. Each phase built on the previous and produced a testable increment before moving forward.

Phase 01 — Requirements & Design

- Defined the core security requirements: confidentiality (encryption), integrity (hashing), and accountability (audit log).
- Evaluated cryptographic algorithms: selected AES-256-CBC as the primary recommendation, 3DES for legacy comparison, and ChaCha20 as a modern stream cipher alternative.

- Designed the binary .enc bundle format to be self-describing (no external manifest needed).
- Sketched the UI layout: two-column index page (encrypt / decrypt), result page, and audit log table.
- Chose Flask over Django for its lightweight nature, appropriate for a focused academic project.

Phase 02 — Cryptography Core

- Implemented all three encryption algorithms using PyCryptodome.
- Implemented PBKDF2-HMAC-SHA256 key derivation with 100,000 iterations.
- Implemented SHA-256 integrity hashing using Python's hashlib.
- Wrote the binary bundle read/write functions (struct.pack / struct.unpack).
- Unit-tested encrypt then decrypt cycles for all three algorithms, verifying round-trip correctness.
- Verified that wrong-password decryption raises ValueError or produces mismatched hash.

Phase 03 — Flask Backend

- Implemented all Flask routes: /encrypt, /decrypt, /verify, /delete, /unlock, /audit, /audit/clear.
- Integrated crypto.py into route handlers with proper error handling.
- Implemented audit.py: SQLite schema creation, INSERT on every operation, SELECT for the log page.
- Implemented logout.py: per-file attempt tracking, automatic lock after 5 failures, manual unlock.
- Tested each route independently using Flask's test client before connecting to the frontend.

Phase 04 — Frontend & User Experience

- Built Jinja2 templates with template inheritance from base.html.
- Designed the CSS stylesheet: Berry & Cream colour palette, Calibri/Cambria typography.
- Implemented the toast notification system (JavaScript) for flash messages.
- Added dynamic algorithm info badges (JavaScript) that update when the user changes the algorithm selector.
- Built the audit log table with client-side filter buttons (All / Encrypt / Decrypt / Verify / Failed / Tampered / Blocked).
- Ensured the vault contents table shows file name, lock status, attempt pips, and delete button.

Phase 05 — Testing & Hardening

- End-to-end testing: encrypted files with each algorithm, decrypted back, verified file contents were identical.
- Lockout testing: submitted 5 wrong passwords, verified file locked; verified unlock works from audit log.
- Tamper testing: manually edited a .enc file with a hex editor, verified tamper detection triggered.
- Edge case testing: empty files, very large files, filenames with special characters.

- Cleaned up code: removed unused imports, standardised error messages, verified all audit log entries were correct.

Summary Timeline

Phase	Focus	Key Deliverable
01 — Design	Architecture & algorithm selection	Binary bundle spec, UI wireframe
02 — Crypto	Core encryption/decryption logic	crypto.py, unit tests
03 — Backend	Flask routes, audit, lockout	app.py, audit.py, lockout.py
04 — Frontend	Templates, CSS, JavaScript	All .html files, style.css
05 — Hardening	Testing, edge cases, polish	Verified working application